Kalman Filter Drone Exercise

Estimating Altitude from Real ArduPilot Flight Data

1. Introduction

This exercise applies a Kalman filter to real drone flight data from an ArduPilot log. The objective is to estimate the drone altitude by combining an IMU acceleration prediction model with different measurement updates. The state vector used throughout the work is x = [altitude; vertical_velocity].

Part 1 uses only barometer altitude as the measurement. Part 2 adds GPS altitude as a second altitude measurement. Part 3 adds GPS vertical velocity so that both altitude and velocity can be measured directly. Part 4 reflects on the measurement matrix H, the Kalman gain K, covariance evolution during a GPS outage, and the handling of sensors with different sampling rates.

The ArduPilot EKF altitude estimate, POS.RelHomeAlt, is used as the reference for comparison. Since different altitude sources can use different offsets or reference datums, the comparison signals are shifted to the same initial barometer altitude reference.

2. Data and filter setup

The main logged signals used are IMU acceleration, aircraft attitude, barometer altitude, GPS altitude, GPS vertical velocity, and ArduPilot POS.RelHomeAlt. IMU timestamps form the prediction time base because the IMU is the highest-rate signal. Barometer and GPS measurements are applied only when their real timestamps arrive.

Quantity	Value
Common flight interval used	312.48 s
IMU prediction samples	56,855
Barometer samples in log	1,953
GPS samples in log	1,010
POS/EKF samples in log	2,033
Initial barometer altitude reference	-0.361 m
Initial covariance P0	diag([0.5, 0.1])
Acceleration process noise sigma_a	0.30 m/s^2
R_baro from stationary window	0.160278 m^2
R_gps_alt from stationary window	0.007347 m^2
R_gps_vz from stationary window	0.033680 (m/s)^2

The vertical Kalman filter uses a two-state constant-acceleration model over each small IMU time step:

```
x_k = F*x_(k-1) + B*a_z
F = [1  dt; 0  1]
B = [0.5*dt^2; dt]
```

The process noise covariance is calculated from the acceleration noise standard deviation:

```
Q = sigma_a^2 * [dt^4/4  dt^3/2; dt^3/2  dt^2]
```

The acceleration input is obtained from the IMU Z-axis acceleration after a simple roll and pitch tilt correction. This is not as complete as a full inertial navigation solution, but it is suitable for demonstrating the Kalman filter prediction and update structure.

3. Part 1: Barometer-only Kalman filter

3.1 Method

The barometer-only filter uses IMU vertical acceleration as the control input in the prediction step and BARO.Alt as the only measurement in the update step. The barometer directly measures altitude, but it does not directly measure vertical velocity. Therefore, the measurement matrix is:

```
z_baro = altitude
H_baro = [1  0]
```

The initial covariance was set to P0 = diag([0.5, 0.1]). This gives moderate uncertainty in the initial altitude and velocity without allowing the first few measurements to dominate unrealistically. The process noise parameter was set to sigma_a = 0.30 m/s^2, which allows the filter to respond to acceleration changes while limiting the effect of acceleration noise.

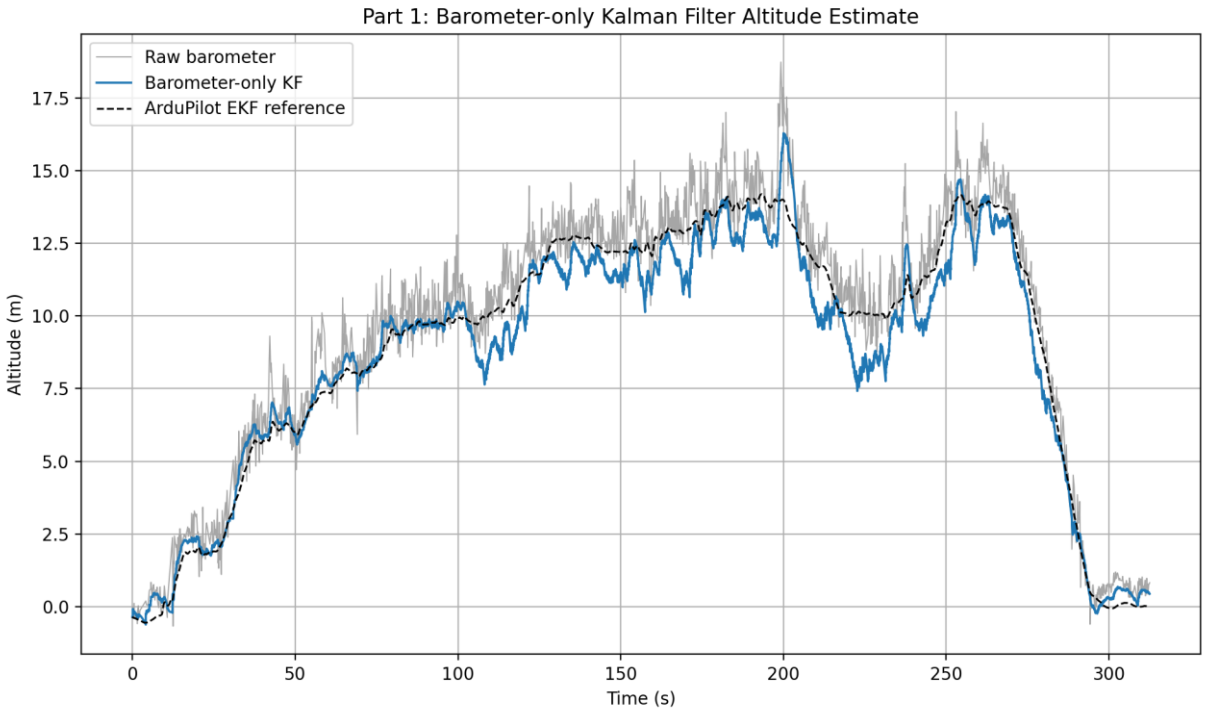

Figure 1. Barometer-only Kalman filter compared with raw barometer altitude and the ArduPilot EKF reference.

3.2 Barometer measurement noise

The starter value for the barometer measurement noise was R_baro = 0.0234 m^2. A data-based value was also calculated from a stationary-looking period in the first 10 seconds of the log. The sample variance of BARO.Alt over this window was R_baro = 0.160278 m^2.

R value	Value	Interpretation
Original hand-selected value	0.0234 m^2	Smaller value; the filter trusts the barometer more strongly.
Empirical stationary-window variance	0.160278 m^2	Larger value based on the actual log, sensor installation, vibration environment, and measurement conditions.

The empirical value is the better choice for this data set because it is estimated from the same sensor and flight log used by the filter. A datasheet value can describe the ideal sensor performance, but it may not include installation effects, vibration, propeller wash, temperature changes, or vehicle motion.

3.3 Results

The raw barometer signal follows the overall altitude profile but contains measurement noise. The Kalman estimate is smoother than the raw barometer signal because it combines the barometer updates with the dynamic prediction from IMU acceleration. The barometer-only estimate follows the ArduPilot EKF reference reasonably well for much of the flight. The largest differences occur during sharper manoeuvres and later in the flight, where acceleration modelling error, sensor bias, or unmodelled vertical dynamics can cause the simple two-state model to diverge from the EKF reference.

4. Part 2: Adding GPS altitude

4.1 Method

The second filter adds GPS altitude as an additional altitude measurement. The state vector and prediction model remain unchanged. Since both the barometer and GPS altitude measure altitude, both scalar measurement updates use the same measurement matrix H = [1 0]. The two sensors are not forced onto a common sampling rate. Instead, each measurement is applied when its timestamp falls between the previous and current IMU prediction timestamps.

```
Barometer update:    z = BARO.Alt,          H = [1  0], R = R_baro
GPS altitude update: z = GPS.Alt_aligned,   H = [1  0], R = R_gps_alt
```

The GPS altitude was aligned to the initial barometer altitude reference before fusion. This alignment is needed because GPS altitude and barometer altitude can use different reference levels.

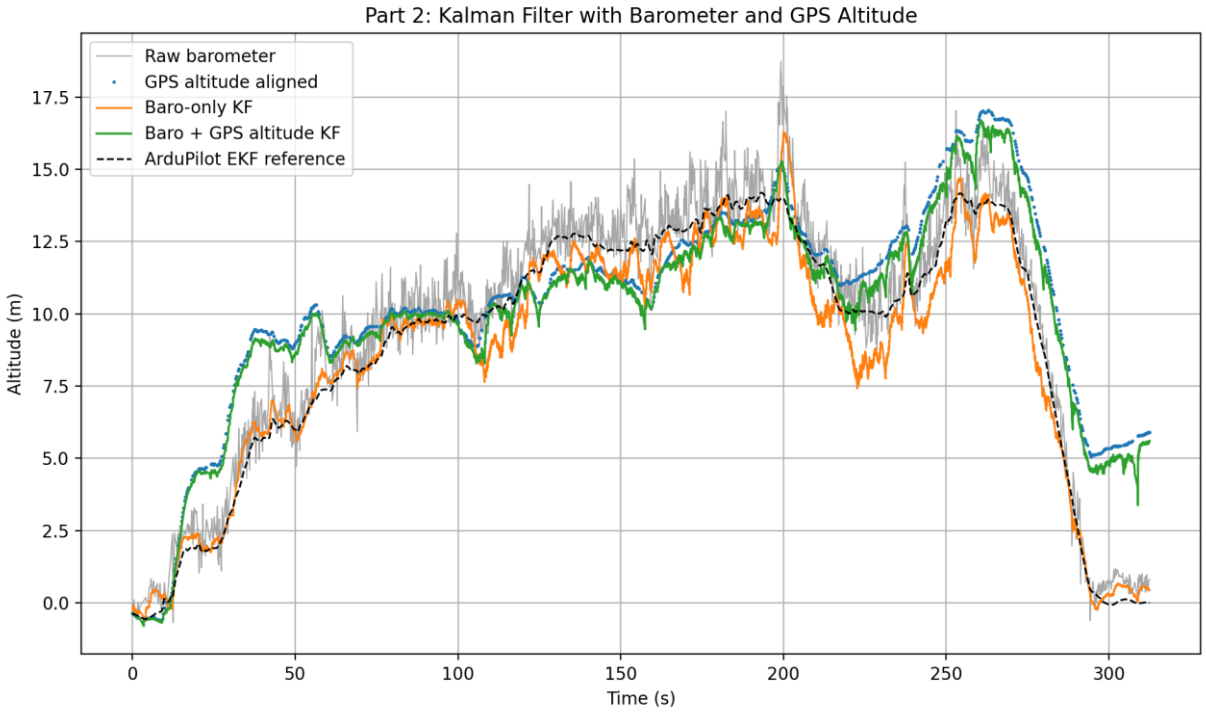

Figure 2. Barometer-only filter and barometer + GPS altitude filter compared with the EKF reference.

4.2 Results

The GPS-aided altitude estimate follows the aligned GPS altitude more strongly than the barometer-only estimate. In principle, GPS altitude can help reduce long-term barometer drift because it provides an independent altitude measurement. In this log, however, adding GPS altitude increased the RMSE against the ArduPilot EKF reference from 0.820 m for the barometer-only filter to 2.117 m for the barometer + GPS altitude filter.

Estimate	RMSE vs EKF reference	Comment
Barometer-only Kalman filter	0.820 m	Closest estimate to the ArduPilot EKF reference for this data set.
Barometer + GPS altitude Kalman filter	2.117 m	Higher RMSE than barometer-only because the very small GPS altitude variance caused the filter to over-trust GPS. This is a tuning lesson, not a filter failure.
Barometer + GPS altitude + GPS.VZ Kalman filter	2.116 m	Similar altitude error to Part 2, but the velocity state is corrected directly using GPS.VZ.

This result should be interpreted as a tuning lesson rather than a filter failure. The GPS altitude stationary-window variance was estimated as 0.007347 m^2, which is much smaller than the empirical barometer variance. As a result, the filter placed strong trust in GPS altitude. If GPS altitude contains bias, delay, multipath error, datum mismatch, or drift, this over-trusting of GPS can pull the filtered estimate away from the EKF reference. A more robust version could use a larger GPS altitude noise value, a longer and more representative stationary window for R estimation, innovation-based tuning, or an additional bias state.

5. Part 3: Adding GPS vertical velocity

5.1 Method

The third filter adds GPS.VZ as a direct measurement of vertical velocity. GPS altitude and GPS vertical velocity arrive in the same GPS message, so they are fused together as one vector measurement update. This is appropriate because the two GPS measurements share the same timestamp.

```
z_gps = [GPS altitude; GPS vertical velocity]
H_gps = [1  0; 0  1]
R_gps = diag([R_gps_alt, R_gps_vz])
```

The first row of H_gps selects altitude from the state vector, while the second row selects vertical velocity. The GPS.VZ sign is adjusted so that the filter convention is positive upward velocity.

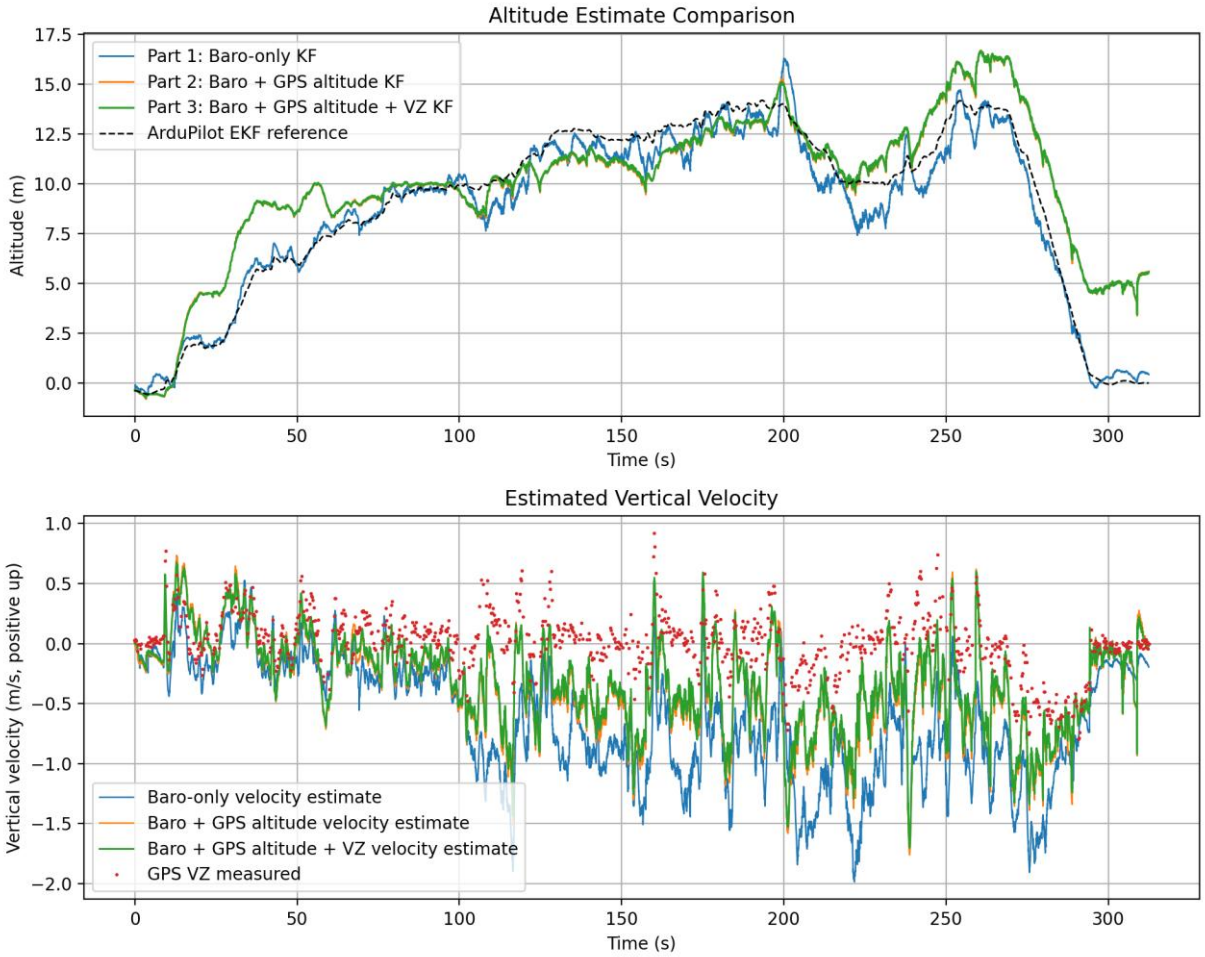

Figure 3. Comparison of all three altitude filters and the corresponding vertical velocity estimates.

5.2 Results

The altitude estimate from Part 3 is similar to the Part 2 estimate because GPS altitude still dominates the altitude correction. The main difference appears in the velocity estimate. In Parts 1 and 2, vertical velocity is inferred indirectly from altitude updates and acceleration prediction. In Part 3, the GPS vertical velocity measurement directly corrects the velocity state. This makes the velocity estimate more directly measurement-informed, although the altitude RMSE changes only slightly compared with Part 2.

6. Part 4: Reflection

6.1 Measurement matrix H

The measurement matrix H maps the state vector x = [altitude; vertical_velocity] into the measurement space. It determines which part of the state each sensor observes.

Part	Sensors used	H matrix	Dimensions	Physical meaning
Part 1	Barometer altitude only	[1 0]	1 x 2	The barometer measures altitude only. It does not directly measure velocity.
Part 2	Barometer altitude and GPS altitude	Each scalar update uses [1 0]	1 x 2 for each scalar update	Both sensors measure altitude. Separate scalar updates are used because the measurements arrive at different times and have different noise values.
Part 3	Barometer altitude plus GPS altitude and GPS.VZ	Baro: [1 0] GPS: [1 0; 0 1]	Baro: 1 x 2 GPS: 2 x 2	The GPS vector update measures both altitude and velocity. Row 1 selects

				altitude; row 2 selects velocity.

As more measured quantities are added, H gains additional rows. H does not change the state vector; it only defines how the current state prediction is compared with the available measurements.

6.2 Kalman gain K

The Kalman gain controls how strongly the prediction is corrected using the measurement innovation. Its dimensions are determined by the number of states and the number of measurements:

```
K dimensions = number of states x number of measurements
```

Part	Measurement dimension	K dimensions	Interpretation
Part 1	1 measurement: barometer altitude	2 x 1	The first entry corrects altitude. The second entry corrects vertical velocity indirectly from altitude innovation.
Part 2	One scalar altitude update at a time: barometer altitude or GPS altitude	2 x 1 for each update	Each altitude measurement can correct both altitude and velocity, depending on the covariance P and measurement noise R.
Part 3	Barometer scalar update and GPS two-value update	Baro: 2 x 1 GPS: 2 x 2	For GPS, column 1 shows how altitude innovation changes the state, and column 2 shows how velocity innovation changes the state.

In Part 1, if the first entry of K is large compared with the second, the barometer innovation mostly changes the altitude estimate and only weakly changes the velocity estimate. This is expected because the barometer measures altitude directly but only informs velocity indirectly through the coupling in the covariance matrix. In Parts 2 and 3, the entries can still be interpreted physically, but the interpretation becomes more coupled when several measurements are fused.

6.3 Covariance evolution and GPS outage

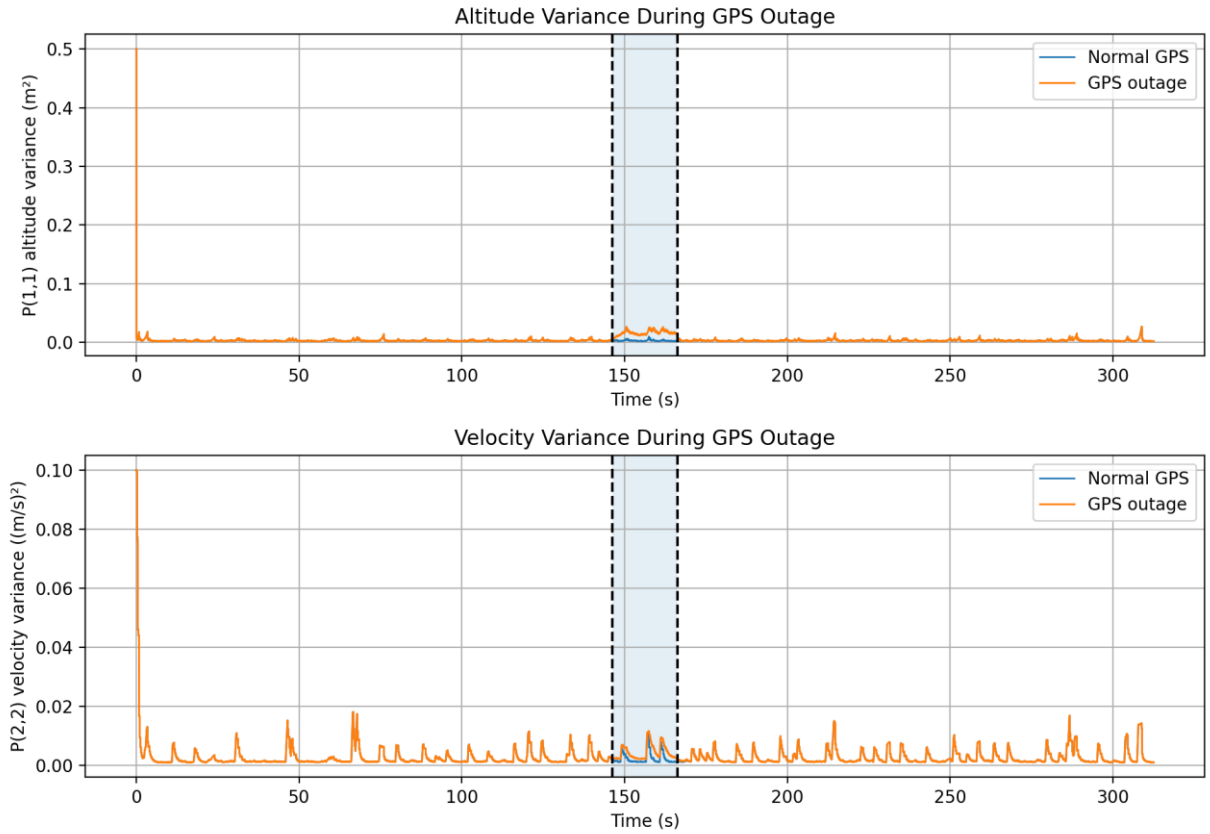

Figure 4. Diagonal entries of the covariance matrix P during a 20 second artificial GPS outage. The shaded region marks the outage window.

A 20 second GPS outage was introduced from approximately 146.2 s to 166.2 s. The plot shows P(1,1), the altitude variance, and P(2,2), the vertical velocity variance. During prediction, uncertainty increases according to P = F*P*F^T + Q. During a measurement update, uncertainty is reduced according to P = (I - K*H)*P because the measurement provides information about the state.

During the GPS outage, the barometer still provides altitude measurements, so the covariance does not grow without bound. However, the covariance is higher during the outage than in the normal GPS case because one measurement source is missing. In this run, the maximum altitude variance inside the outage window is approximately 0.02637 m^2, and the maximum velocity variance is approximately 0.01164 (m/s)^2. When GPS measurements return, the covariance decreases again because GPS altitude information is restored.

6.4 Multi-rate sensor handling

The implementation uses an event-based update strategy to handle sensors with different sampling rates. The IMU timestamps are used as the prediction timeline. At every IMU prediction step, the code checks whether any barometer or GPS measurement timestamp falls between the previous and current IMU timestamp. If a measurement is available, it is processed once and the corresponding sensor index is advanced.

This approach is appropriate because it uses each real measurement at its own timestamp. It avoids nearest-neighbour lookup, which can reuse samples or use future information, and it avoids interpolation to a common rate, which would create artificial measurements. In Part 2, barometer and GPS altitude are processed as separate scalar updates because they do not usually arrive at exactly the same time. In Part 3, GPS altitude and GPS.VZ are fused in one vector update because they come from the same GPS message and share the same timestamp.

7. Conclusion

The barometer-only Kalman filter produced the closest altitude estimate to the ArduPilot EKF reference for this data set, with an RMSE of 0.820 m compared with 2.117 m for the barometer + GPS altitude filter. This does not mean that GPS fusion failed; it shows the importance of measurement-noise tuning and reference alignment in a multi-sensor Kalman filter. The GPS altitude variance estimated from the stationary window was only 0.007347 m^2, so the filter over-trusted GPS altitude. The result is therefore a useful tuning lesson: additional sensors improve a filter only when their noise, bias, timing, and reference frame are modelled appropriately.

The GPS vertical velocity update in Part 3 directly corrected the velocity state, while the GPS outage analysis in Part 4 showed the expected covariance behaviour: uncertainty increases when a measurement source is removed and decreases again when measurements return. The event-based update method is suitable for asynchronous IMU, barometer, and GPS data because it applies each measurement at its own logged timestamp.